TELIPORT SEASON 4 · CASE STUDY 2 · TATA ELXSI

BatteryX AI

Project documentation: architecture and backend working

Reusable AI-assisted digital platform for designing battery energy systems



Change one parameter. See every consequence.

Team Bit Manipulators

Amey Mohite · Harshad Khatale · Shambhu Gaikwad

Walchand College of Engineering, Sangli

Version 0.4.0 · 9 November 2026 · All results are model estimates, not measurements; rule checks are not a safety certification.

CONTENTS

Table of contents

1. Project overview 3

1.1 Four design principles . . . 3

1.2 Scope and key figures . . . 3

2. Repository structure 4

2.1 Technology stack . . . 4

3. System architecture 5

3.1 Why this shape . . . 5

4. Frontend 6

4.1 State and data flow . . . 6

4.2 Live simulator (merged stage-1 prototype) . . . 6

5. Backend: how it works 8

5.1 Start-up (app/main.py) . . . 8

5.2 Life of one request . . . 8

5.3 The common module result . . . 8

6. Engine pipeline and models 9

6.1 Electrical simulation in detail . . . 10

6.2 Thermal model limits . . . 10

7. BMS rule engine and comparison 10

7.1 Constraint-first comparison . . . 10

7.2 Result on the stage-1 designs . . . 11

8. AI integration 12

8.1 Five capabilities . . . 12

8.2 Tools Claude can call . . . 12

8.3 Verifier checks (app/ai/verifier.py) . . . 13

9. Data storage and traceability 13

9.1 Traceability chain . . . 13

9.2 Configuration (backend/.env) . . . 14

10. API reference 14

10.1 Errors . . . 15

11. Running, testing and verification 15

11.1 What the tests prove . . . 15

12. Limitations and roadmap 16

WHAT AND WHY

1. Project overview

BatteryX AI is a web platform that takes a small battery module from requirements to a traceable engineering report. An engineer picks a cell from a database, chooses a series-parallel topology, runs electrical and thermal simulations, checks protection and monitoring rules, compares alternatives, asks an AI copilot to explain the results, and exports a report. Every number shown can be traced back to its inputs, the model that computed it and the assumptions behind it.

The platform answers TELIPORT Season 4, Case Study 2 (Tata Elxsi): *a reusable digital platform for designing battery energy systems*. The core difficulty in that brief is coupling: one change, such as faster charging, ripples across electrical, thermal, safety, mechanical and cost at once. BatteryX makes those ripples visible before any hardware is built.

1.1 Four design principles

Principle	What it means in the code
Physics computes	Every engineering number comes from versioned, deterministic engine functions in `backend/app/engine/`. The same functions serve the API, the AI tools and the tests.
Rules check	15 deterministic safety checks compare results with datasheet or engineer-set limits and record the source of each limit.
AI explains	Claude may only call engine and database tools. A backend verifier rejects any number, reference or claim that is not grounded in tool results.
Engineer decides	Every input carries a source tag; every result is labelled ESTIMATE; reports list assumptions, gaps and limitations.

1.2 Scope and key figures

Item	Value
Demonstration scope	Low-voltage 4S LFP solar-storage module (platform accepts up to 32S × 16P)
Frontend	13 pages (12 engineering pages + Live simulator), React 18 + Vite + Tailwind
Backend	FastAPI (Python 3.11), about 4,150 lines of application code, API v0.4.0
Engine	11 versioned engineering models, 15 safety checks + max safe charge current
AI	5 copilot capabilities, 9 engine/database tools, answer verifier with one repair round
Storage	MongoDB Atlas, with automatic SQLite / PostgreSQL fallback
Tests	109 backend (pytest) + 44 frontend (vitest) + real-browser workflow (14/14 steps)
Speed	about 0.04 s per full design evaluation; 0.11 s for an A/B/C comparison (laptop)

Honesty note. All results are model estimates, not measurements. The only cell in the database is the stage-1 *illustrative* example (not a real product). The rule checks screen a design; they are not a safety certification.

WHERE THINGS LIVE

2. Repository structure

The project is one repository, `batteryx/`, with three top-level folders. The stage-1 BatteryWeave prototype was merged into the frontend as the Live simulator, so there is no second app.

```
batteryx/
├── README.md
├── backend/                      FastAPI service (Python)
│   ├── app/
│   │   ├── main.py               app factory: settings, database choice, routers, CORS, /health
│   │   ├── config.py             .env settings (keys are redacted in logs and /health)
│   │   ├── errors.py             exception → {"error": {code, message, details}} mapping
│   │   ├── ratelimit.py          per-minute limit for AI endpoints
│   │   ├── api/                  routes.py · routes_extra.py · routes_copilot.py
│   │   ├── schemas/              cell.py · design.py · project.py · results.py   (Pydantic)
│   │   ├── engine/               11 models + pipeline.py + comparison.py + registry.py
│   │   ├── ai/                   claude_client · copilot · copilot_tools · verifier · grounding · llm
│   │   ├── db/                   database.py · repository.py · projects.py · conversations.py
│   │   └── data/                 seed_cells.json · example_designs.json
│   ├── tests/                    109 pytest tests
│   ├── scripts/                  demo_scenarios.py · capture_fixtures.py · copilot_smoke.py
│   └── docs/                     API.md · ENGINEERING_MODELS.md · COPILOT.md
├── frontend/                     React + Vite + Tailwind
│   ├── src/
│   │   ├── App.jsx               routes
│   │   ├── pages/                13 pages (Projects … Report, Simulator)
│   │   ├── components/           layout · ui · charts · cells · design · three (3D)
│   │   ├── store/project.js      zustand store: project, active design, results, autosave
│   │   ├── lib/                  api.js · design.js · fields.js · format.js · report.js
│   │   ├── simulator/            merged stage-1 engine, charts and scoped CSS
│   │   └── test/                 44 vitest tests + backend fixtures
│   └── e2e/                      Puppeteer real-browser workflow + screenshot scripts
└── docs/                         technical docs, audit, demo results, screenshots, presentation
```

2.1 Technology stack

Layer	Technologies
Frontend	React 18.3, Vite 5, Tailwind CSS 3, React Router 6, zustand (state), Recharts (charts), Three.js via @react-three/fiber and drei (3D), lucide-react icons, react-markdown
Backend	Python 3.11, FastAPI, Pydantic v2, Uvicorn, NumPy, SciPy (brentq root finding)
Data	pymongo (MongoDB Atlas), SQLAlchemy (SQLite / PostgreSQL)
AI	Official anthropic SDK (Claude, server-side), optional Groq client
Testing	pytest, vitest + Testing Library + jsdom, Puppeteer-core with Chrome

THE BIG PICTURE

3. System architecture

BatteryX uses three simple layers: a browser application, a FastAPI backend, and external services (database and Claude). The backend is split internally into an API layer, a pure engine, an AI layer and a data layer.

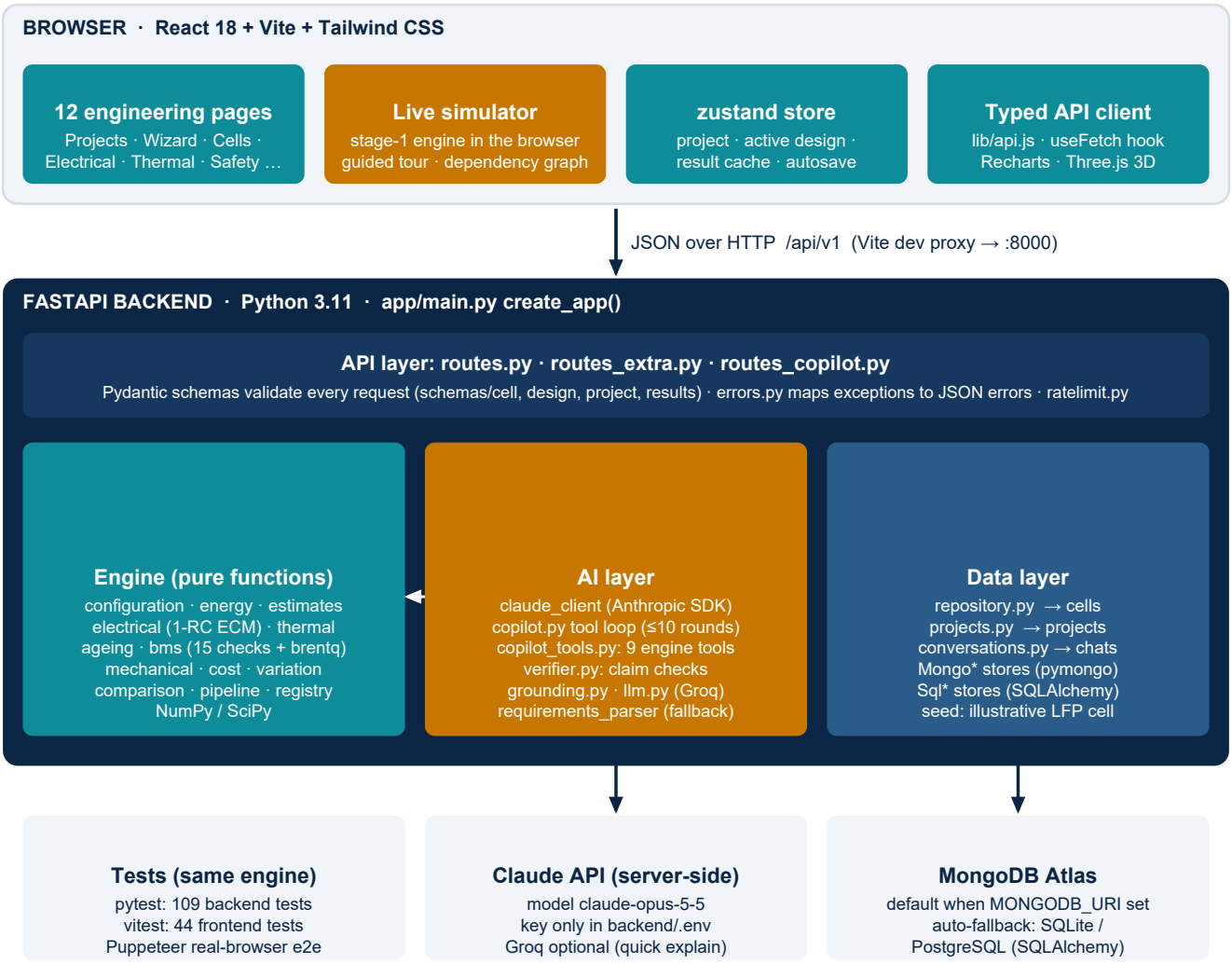


Figure 1. Layered architecture. Orange marks the parts that are not pure server-side physics (the in-browser simulator and the AI layer).

3.1 Why this shape

- **A pure, deterministic engine.** Engine functions take a `CellSpec` and a `DesignInput` and return results; they never touch the database, network or clock. That makes them fast, testable, and safe to expose to the AI as tools.
- **One design document.** `DesignInput` is the only interface between modules. Each page edits part of it; each model reads the parts it needs.
- **Swappable storage.** Stores share one interface with Mongo and SQL implementations, chosen at start-up.
- **Keys stay on the server.** The browser never calls Claude or the database directly; it only calls `/api/v1`.

WHAT THE USER SEES

4. Frontend

The frontend is a single-page React app. `AppShell` provides the sidebar, a design switcher (project / active design / topology / data-status badge / save state) and a backend-status light. Each page renders inside an error boundary, so one failing page never takes down the app.

Group	Page (route)	What it does	Backend call
Workspace	Projects (/app/projects)	Create, open, evaluate all designs of a project	POST/GET /projects
	Configuration wizard (/app/wizard)	Step-by-step design entry with source tags	/configuration, /energy, /designs/evaluate
	Live simulator (/app/simulator)	Guided 5-step tour, heat map, dependency graph	none (in-browser model)
Design	Cell selection (/app/cells)	Cell database with provenance; add datasheet cells	/cells
	Series-parallel (/app/topology)	S×P sizing, live pack figures	/configuration, /energy
Analysis	Electrical (/app/electrical)	CC-CV charge and CC/CP discharge time series	/simulate/charge, /simulate/discharge
	Thermal (/app/thermal)	Transient curve, current sweep, simplified label	/thermal, /estimates
	BMS & safety (/app/safety)	15 checks, max safe current, Monte-Carlo	/bms, /variation
	Mechanical packaging (/app/packaging)	Envelope, mass breakdown, busbars, 3D view	/mechanical, /cost
Decide	Design comparison (/app/compare)	Constraint-first ranking, deltas, recommendation	/designs/compare
	AI assistant (/app/assistant)	Claude copilot in 5 modes, verified answers	/copilot/*, /ai/*
	Engineering report (/app/report)	Print/PDF, Markdown, JSON with full traceability	/designs/evaluate, /designs/compare, /models

4.1 State and data flow

- **zustand store** (`store/project.js`) holds the open project, the active design name, a result cache keyed by design, and the save state. Edits mark the project dirty and autosave it with `PUT /projects/{id}`; the last project is restored on reload.
- **API client** (`lib/api.js`) wraps fetch, decodes the JSON error envelope, and is used through the `useFetch` hook.
- **No physics in the browser** for project designs: forms edit the design, buttons call the backend, charts render the response. PASS / WARN / FAIL statuses come from the backend rule engine.

4.2 Live simulator (merged stage-1 prototype)

The Live simulator is the only page that computes in the browser. It runs the stage-1 closed-form model (`src/simulator/model/engine.js`) so the guided walkthrough and dependency graph react instantly and work even when the backend is offline. A test pins it to the same A/B/C numbers as the backend parity test, so the two engines cannot drift apart. Its CSS is scoped under `.bw` so it cannot clash with Tailwind class names.

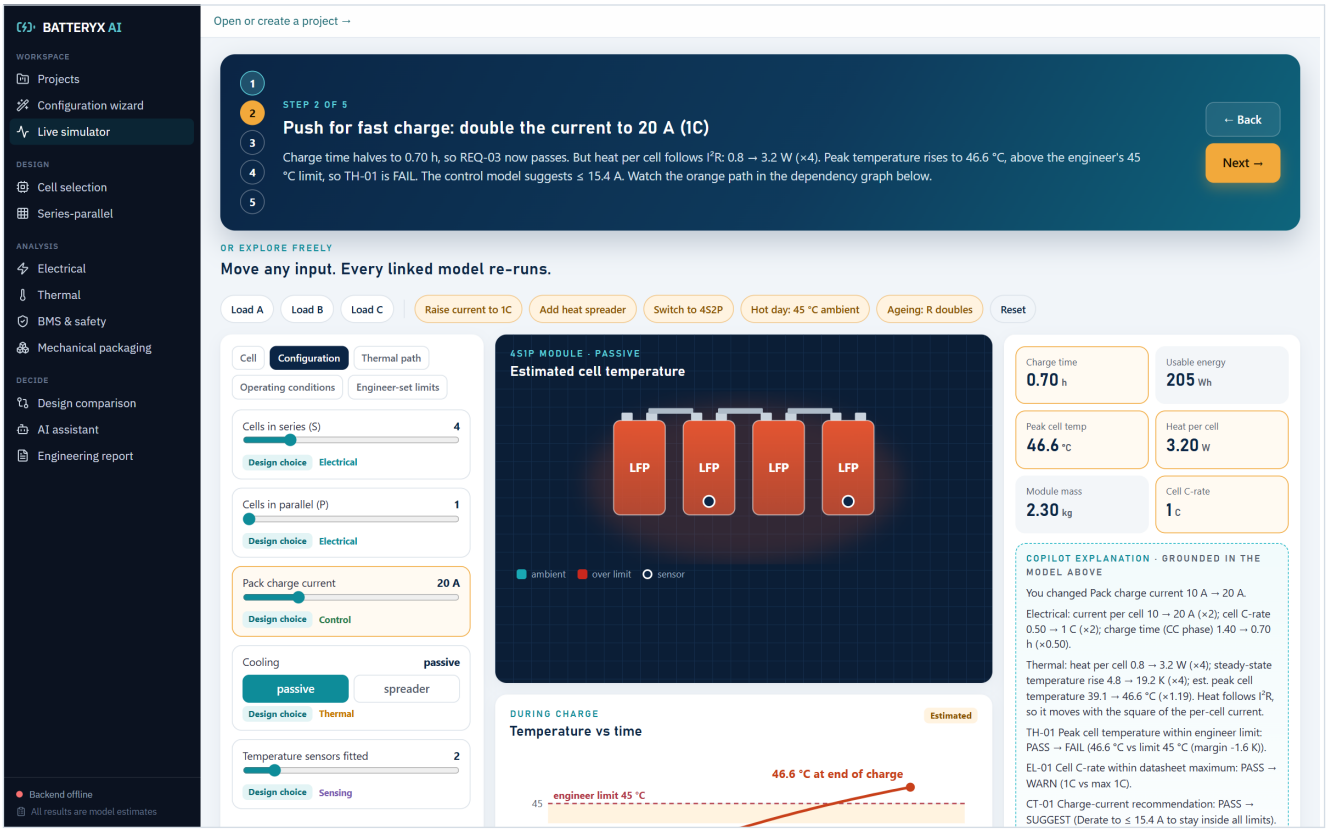


Figure 2. Live simulator, guided step 2: raising the charge current to 20 A trips TH-01; the explanation is generated from the model's own numbers.

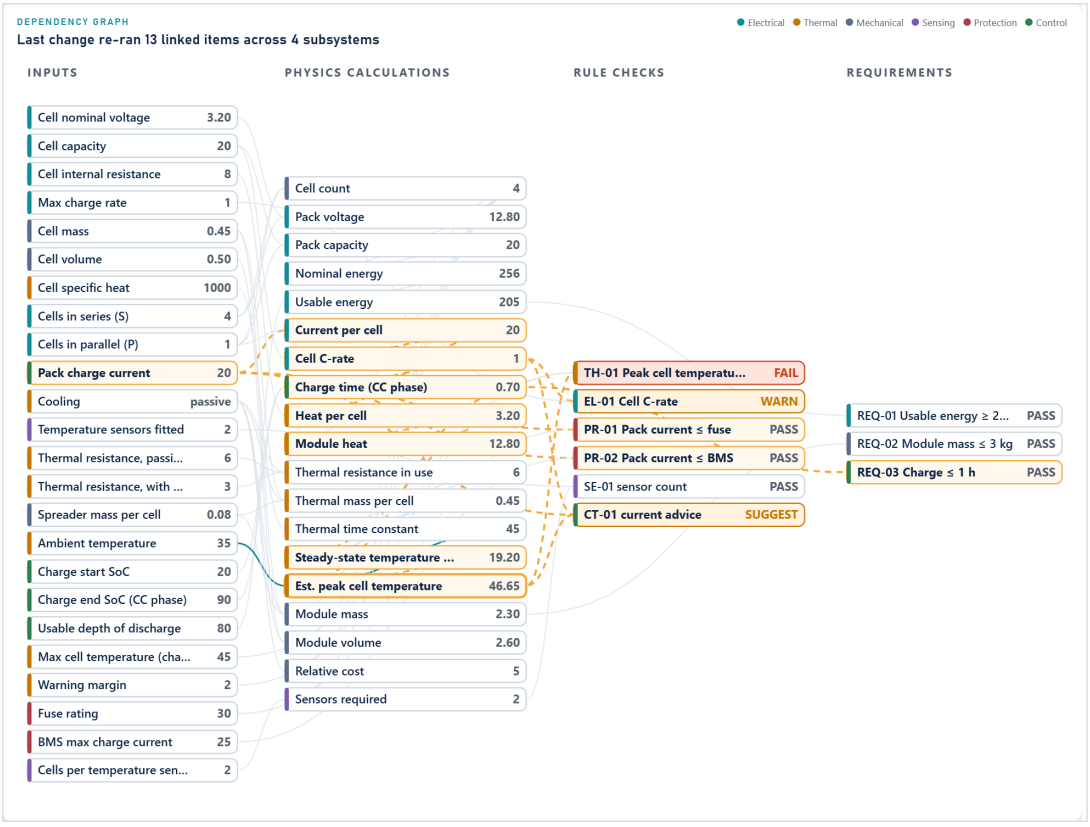


Figure 3. Dependency graph: a 10 A → 20 A change re-runs 13 linked items across 4 subsystems (orange path).

FROM REQUEST TO RESULT

5. Backend: how it works

5.1 Start-up (app/main.py)

- **Load settings** from `backend/.env` (`config.py`). Secrets are redacted wherever they could be printed.
- **Choose storage.** With `BATTERYX_DB_BACKEND=auto` and a `MONGODB_URI`, the app pings MongoDB Atlas (8 s timeout). If that fails it logs the reason and falls back to SQL (SQLite by default). With `mongo` it refuses to start instead of falling back.
- **Seed** the illustrative LFP cell if it is missing.
- **Build clients:** `ClaudeClient` (Anthropic SDK) and the optional Groq `LLMClient`; both report "not configured" rather than failing when no key is set.
- **Mount routers** (`routes`, `routes_extra`, `routes_copilot`), install error handlers and CORS, and expose `/health` (status, version, active database, fallback reason; never credentials).

5.2 Life of one request

The most important call is `POST /api/v1/designs/evaluate`. Every other single-module endpoint follows the same path but runs one model.

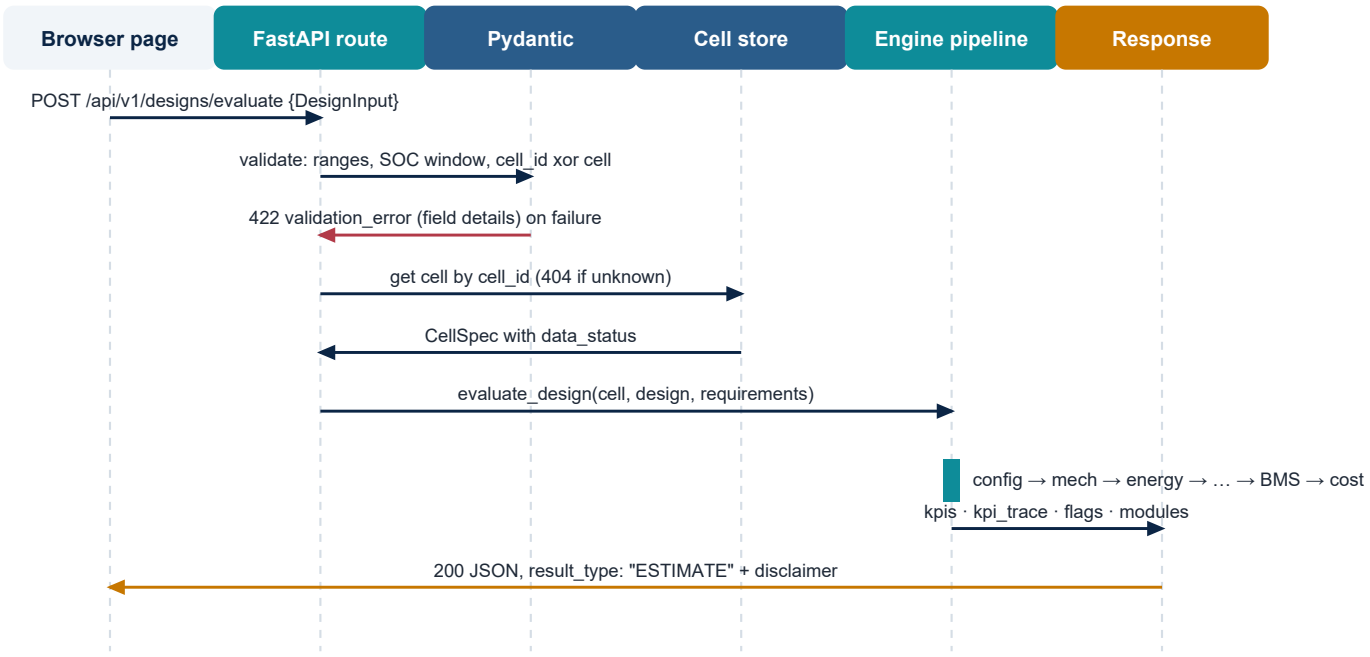


Figure 4. Sequence of a design evaluation. Red: validation failure path. Orange: the response returned to the page.

- **Validation** (Pydantic): ranges (S 1–32, P 1–16), `soc_min < soc_max`, charge start < target, not both discharge current and power, exactly one of `cell_id` or inline `cell`, `datasheet_ref` required for datasheet cells, strictly increasing OCV table.
- **Engine**: `evaluate_design()` runs every module in dependency order (next section).
- **Response**: KPIs with units, `kpi_trace`, requirement statuses, safety summary, every module's full result and flags, and the echoed inputs with their source tags.

5.3 The common module result

Every engine module returns the same envelope, so the frontend, report and AI can treat all modules alike:

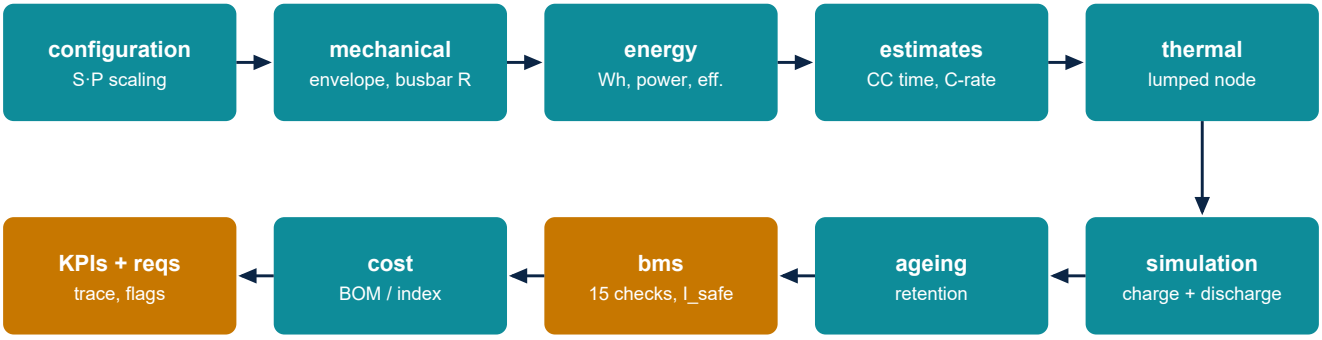
```
{ "module": "thermal", "result_type": "ESTIMATE",
  "model":  { "id", "version", "fidelity", "label", "equations": [...], "assumptions": [...] },
  "values": { ... numbers with units in their field names: _v, _a, _wh, _c, _kg ... },
  "flags":  [ { "severity", "code", "message", "field", "module" } ] }
```

`null` means "not computable from the given inputs", and a flag always says why. Flag severities: `missing_input`, `unsupported`, `assumption`, `warning`, `info`. Missing datasheet data is flagged, never invented.

THE PHYSICS

6. Engine pipeline and models

evaluate_design() in app/engine/pipeline.py: one DesignInput in, one evaluation dict out



Interconnect resistance from mechanical feeds energy and simulation; estimates feed thermal; simulations feed the BMS voltage checks.

Figure 5. Order of evaluation inside `evaluate_design()`*. If the design has an ageing assumption, the charge and discharge simulations are re-run with aged capacity and resistance.*

Notation: S series, P parallel, N = S·P cells, Q cell capacity [Ah], R0 ohmic resistance [Ω], I current [A], w = $(SOC_{max} - SOC_{min})/100$.

Model (id)	Key equations	Fidelity
Configuration (series_parallel)	$V_{pack} = S \cdot V_{cell}$; $Q_{pack} = P \cdot Q$; $R_{pack} = S \cdot R0/P$; $I_{max} = P \cdot Q \cdot C_{max}$	closed form
Energy (nominal)	$E = V_{nom} \cdot Q_{pack}$; $E_{usable} = E \cdot w$; $P = V \cdot I - I^2R$; $\eta = 1 - I \cdot R/V$	closed form
Estimates (closed_form_time)	$t_{charge,CC}$ = ΔSOC / C-rate; $t_{discharge} = Q \cdot w/I$ or E_{usable}/P	closed form
Electrical (ecm_1rc)	1-RC Thevenin, exact zero-order-hold steps (Δt = 1 s): SOC, V1 update; V = OCV(SOC) − I·R0 − V1; CC-CV charge, CC or CP discharge	time domain
Thermal (lumped_single_node)	$m \cdot c_p \cdot dT/dt = q - (T - T_{amb})/R_{th}$; $q = I^2R0$; $\tau = m \cdot c_p \cdot R_{th}$	SIMPLIFIED
Ageing (simplified_assumption)	retention = $1 - (1 - EOL) \cdot N/N_{life}$, or user fade and resistance-growth rates	SIMPLIFIED
BMS (rule_checks)	15 checks; max safe charge current by SciPy brentq on the closed-form temperature, capped by C-rate, fuse and BMS	rule engine
Mechanical (grid_envelope)	$L = cols \cdot f_x + (cols-1) \cdot gap + 2 \cdot wall$; busbar A = I/J, R = ρL/A, $m = \rho_m AL$	closed form
Cost (bom_rollup)	currency roll-up from given prices; relative index $N \cdot (1 + k_{cool}) + k_{BMS}$	closed form
Variation (monte_carlo)	seeded normal sampling of R and Q; current split $I_i = I \cdot G_i/\Sigma G$; worst-cell temperature	statistical
Comparison (constraint_first_weighted)	safety FAIL □ ineligible; min-max normalised weighted score; −0.5 per requirement not met	aggregation

6.1 Electrical simulation in detail

```
SOC[k+1] = SOC[k] − I·Δt / (3600·Q)
V1[k+1]  = V1·exp(−Δt/R1C1) + R1·I·(1 − exp(−Δt/R1C1))
V        = OCV(SOC) − I·R0 − V1            (I > 0 means discharge)
q        = I²·R0 + V1²/R1                   (heat into the thermal node)
CC-CV:   CC until V = V_max, then I = (OCV − V1 − V_max)/R0 until |I| ≤ I_cut (0.05C, flagged)
CP:      solve R0·I² − (OCV − V1)·I + P = 0 each step
Pack:    V_pack = S·V − I_pack·R_interconnect
```

Fallbacks are explicit: without R1/C1 the model is R0-only (flagged as an assumption); without an OCV table the OCV is fixed at nominal, and the CV phase and voltage cut-offs are reported as *not modelled*; without R0 no simulation is run.

6.2 Thermal model limits

The thermal model is a single lumped node, clearly labelled SIMPLIFIED. It does **not** model cell-to-cell conduction, centre-versus-edge cells, internal gradients, entropic heat, temperature-dependent resistance, coolant temperature rise or thermal runaway.

CHECKS AND DECISIONS

7. BMS rule engine and comparison

`engine/bms.py` screens every design against datasheet and engineer-set limits. Each check records its value, limit, limit source and basis. A missing input gives **NOT_EVALUATED**, never PASS.

ID	Check	ID	Check
EL-01	Charge C-rate vs datasheet max	PR-02	Charge current vs BMS limit
EL-02	Discharge C-rate vs datasheet max	PR-03	Discharge current vs BMS limit
VL-01	Simulated max voltage within window	PR-04	Fuse rated above BMS limit (coordination)
VL-02	Simulated min voltage within window	PR-05	Fuse vs cell-group current capability
TH-01	Peak cell temperature on charge vs limit	SE-01	Temperature sensor count vs cells per sensor
TH-02	Peak cell temperature on discharge vs limit	SE-02	Voltage sense channels vs series groups
TH-03	Ambient vs datasheet charge window	BL-01	Passive balancing time $\Delta SOC \cdot Q / I_{bleed}$
PR-01	Pack current vs fuse rating	CT-01	Max safe charge current (advice, brentq)

Statuses: PASS / WARN (within the warning margin) / FAIL / NOT_EVALUATED. These checks screen a design; they do not model fuse time-current curves or BMS response times and are **not a certification**.

7.1 Constraint-first comparison

- **Eligibility first:** any safety FAIL makes a design ineligible, whatever its score.
- **Score:** each metric (usable energy, CC charge time, peak charge temperature, mass, volume, relative cost) is min-max normalised within the set and weighted. Weights can come from the user, or from the AI optimizer translating stated priorities.
- **Penalty:** each requirement NOT_MET subtracts 0.5.
- **Output:** ranking, deltas vs the baseline, trade-offs, risks, model limitations and a recommended next step.

7.2 Result on the stage-1 designs

Estimate	A 4S1P passive	B 4S1P + spreader	C 4S2P
Usable energy [Wh]	204.8	204.8	409.6
CC charge time [h]	1.4	0.7	1.4
Peak cell temperature [°C]	39.06	42.63	39.06
Module mass [kg]	2.30	2.62	4.30
Max safe charge current [A]	15.4	20.0	25.0
Checks WARN / FAIL	0	1 (EL-01 WARN)	0
Requirements missed	REQ-03 (≤ 1 h)	none	REQ-02 (≤ 3 kg), REQ-03
Engine rank	2	1 (recommended)	3

Illustrative stage-1 cell data; model estimates. Charging design A at 20 A raises the estimated peak to 46.65 °C (TH-01 FAIL, limit 45 °C). Reproduced by `backend/scripts/demo_scenarios.py`.

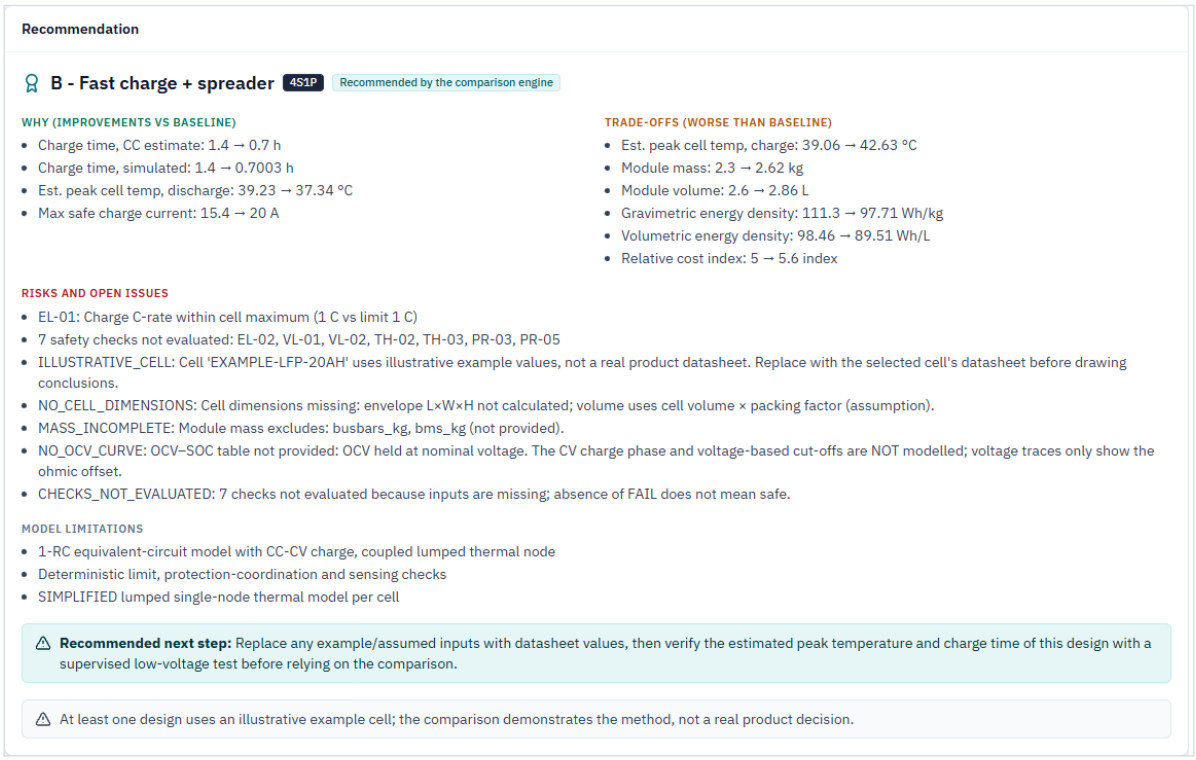


Figure 6. Comparison page: recommendation, trade-offs and risks come from the comparison endpoint.

COPILOT AND VERIFIER

8. AI integration

The copilot turns plain-language questions into engine runs and explains the results. Claude never computes engineering values: it calls BatteryX tools, receives results tagged with references (T1, T2, …), and must answer in a structured JSON format where every number cites a reference. The backend then verifies the answer before the user sees it.

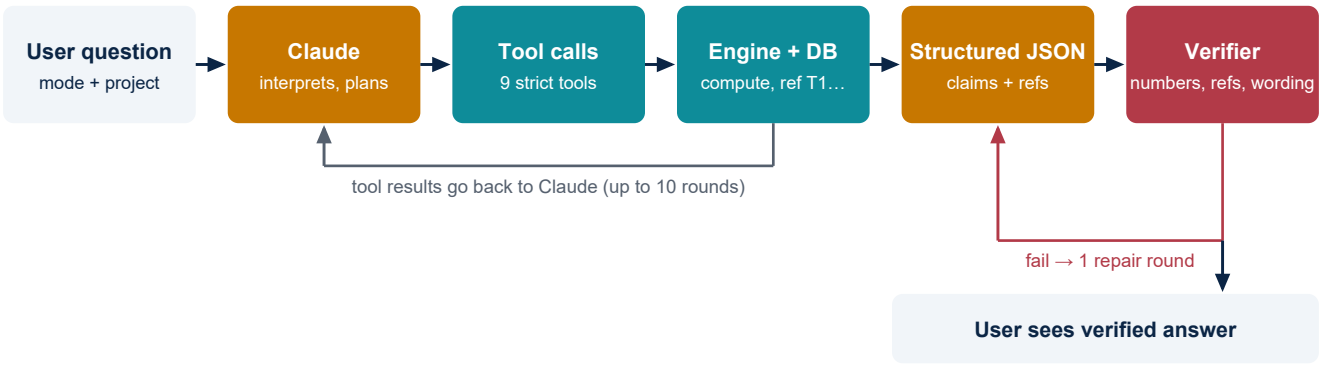


Figure 7. Copilot loop (`app/ai/copilot.py`): manual tool loop with the official Anthropic SDK, at most 10 tool rounds, one repair round on verification failure.

8.1 Five capabilities

Mode	Capability	What Claude must do
design_copilot	Design Copilot	Map requirements to metrics, pick cells only from the database, propose ≤ 3 configurations and evaluate each with the engine
analyst	Engineering Analyst	Run simulations, explain electrical and thermal drivers, cite model fidelity
safety	Safety Assistant	Explain each WARN / FAIL / NOT_EVALUATED and what resolves it; never certify
optimizer	Optimization Advisor	Turn priorities into weights, compare, sweep a parameter, propose engine-checked improvements
report	Report Generator	Write report sections where every number is sourced

8.2 Tools Claude can call

Tool	Backed by
list_cells, get_cell	Cell database (the only source of cell specifications)
get_project_designs	Project store: designs, conditions, assumptions, requirements
evaluate_design	Full engine pipeline on a base design plus whitelisted changes
simulate	1-RC ECM + lumped thermal (charge or discharge)
safety_checks	BMS rule engine, max safe current, binding constraint
compare_designs	Comparison engine with weights from the user's priorities
sweep_parameter	Engine, up to 8 values of series, parallel or current
get_model_documentation	Model registry: equations, assumptions, fidelity

Whitelisted changes only: series, parallel, an existing cell_id, currents or power, SOC windows, sensor counts, and copying cooling or mechanical settings from another design. Claude cannot type in thermal resistances, masses or any other assumption. All tools use strict JSON schemas.

8.3 Verifier checks (app/ai/verifier.py)

Check	Catches
invalid_reference	A claim cites a reference no tool call produced this turn
value_mismatch	A claim's number is not in the cited tool result (e.g. 41.7 °C when the engine said 39.058 °C)
ungrounded_number / unsourced_number	Numbers in no tool result, project input or the question; "general knowledge" figures
candidate_not_evaluated	A proposed design that was never run, or run with different changes
unknown_cell / unverified_reference	Invented part numbers, invented datasheet URLs
certification_claim / measurement_claim	"complies with IEC 62619", "is safe"; estimates presented as measured

The verification result (passed/failed, every issue, numbers checked) is stored with the conversation and shown in the UI; a failed answer is labelled red and its candidates cannot be added to the project. Prompt caching is used for the static system prompt and tool list. Status: the copilot tests pass with a scripted model driving the real engine; the live Claude run needs an API key.

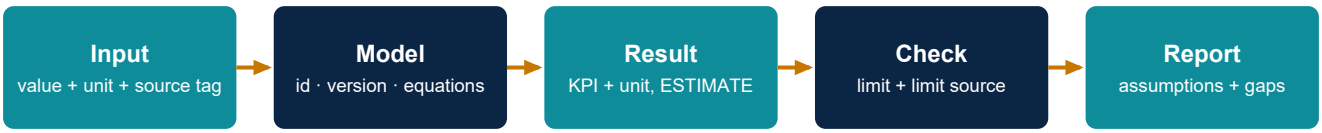
WHAT IS KEPT, AND HOW NUMBERS ARE TRACED

9. Data storage and traceability

Store	Mongo collection / SQL table	Key	Content
Cells	cells	cell id	CellSpec (identity, V_{nom}, Ah, data_status required; all else optional), origin seed/user
Projects	projects	12-hex id	name, application, description, designs[] (DesignInput drafts), requirements[], timestamps
Conversations	conversations	12-hex id	project_id, turns: message, mode, structured answer, verification, tool calls and results

- A cell cannot be deleted while a design uses it; seeded cells cannot be deleted.
- Deleting a project cascades to its conversations.
- Design names are unique within a project. Projects store *drafts*, validated when evaluated.
- `data_status` is one of `datasheet` (requires `datasheet_ref`), `user_entered`, `illustrative_example`.

9.1 Traceability chain



Every KPI in evaluation.kpi_trace names the module, model id, version and fidelity that computed it.

Figure 8. From input to report. The engineering report prints every link: inputs with source tags, the model behind each KPI, check limits and their sources, assumptions, limitations and an equations appendix. AI answers add one more link: claim → source_ref → tool call → engine result.

9.2 Configuration (backend/.env)

Variable	Purpose
MONGODB_URI, BATTERYX_MONGODB_DB	MongoDB Atlas connection and database name
BATTERYX_DB_BACKEND	auto (Mongo, else SQL) · mongo (must connect) · sql
BATTERYX_DATABASE_URL	SQL URL for the fallback (SQLite by default, PostgreSQL supported)
ANTHROPIC_API_KEY, BATTERYX_CLAUDE_MODEL, _EFFORT, _FALLBACKS	Claude copilot (default model claude-opus-5-5)
GROQ_API_KEY, GROQ_MODEL	Optional quick explain
BATTERYX_AI_RATE_PER_MIN	Per-minute limit for AI endpoints
VITE_API_URL (frontend/.env)	API base URL for a deployed build (empty in development)

ENDPOINTS

10. API reference

Base URL `http://localhost:8000`; interactive docs at `/docs`. JSON in, JSON out; units are in field names.

Method	Path	Returns
GET	/health	status, version, active database (no credentials)
GET	/api/v1/models · /metrics · /examples/designs	model equations and assumptions · KPI keys and units · stage-1 designs
GET POST DELETE	/api/v1/cells[/{id}]	cell list / one cell / create (201) / delete user cells (204)
POST	/api/v1/configuration · /energy · /estimates	series-parallel sizing · energy, power, efficiency · closed-form times
POST	/api/v1/simulate/charge · /simulate/discharge	time-domain run + time series
POST	/api/v1/thermal · /ageing	SIMPLIFIED thermal estimate · SIMPLIFIED ageing
POST	/api/v1/bms · /mechanical · /cost	rule checks, max safe current · envelope, mass, busbars · cost
POST	/api/v1/variation	Monte-Carlo percentiles
POST	/api/v1/designs/evaluate	all modules + KPIs + requirements + flags + trace
POST	/api/v1/designs/compare	ranking, deltas, recommendation
GET POST PUT DELETE	/api/v1/projects[/{id}]	project CRUD
GET POST	/api/v1/ai/status · /ai/explain · /ai/extract-requirements	quick grounded explain; requirement extraction (AI or rule-based)
GET POST DELETE	/api/v1/copilot/status · /projects/{id}/chat · /conversations	Claude copilot turns and stored conversations

10.1 Errors

HTTP	code	Meaning
404	not_found	Unknown cell, project or conversation
409	conflict / conversation_mismatch	Duplicate cell, cell in use, seeded-cell delete, wrong project
422	validation_error / engine_input_error / ai_refused	Invalid input (field details), invalid comparison, model refusal
429	rate_limited / ai_rate_limited	Platform or provider limit (with retry_after_s)
502	ai_provider_error	Provider or network error, malformed AI output, tool-loop limit
503	ai_unavailable	No or invalid AI key

All errors use one envelope: `{"error": {"code", "message", "details"}}`. The frontend shows them in an ErrorBox with field-level details.

HOW TO REPRODUCE EVERYTHING

11. Running, testing and verification

```
# backend
cd batteryx\backend
pip install -r requirements.txt
python -m uvicorn app.main:app --port 8000          # API on http://localhost:8000
python -m pytest -q                                 # 109 tests
python -I scripts\demo_scenarios.py                 # regenerate docs\DEMO_RESULTS.md

# frontend
cd batteryx\frontend
npm install
npm run dev                                         # http://localhost:5173 (proxies /api to :8000)
npm test                                            # 44 tests
$env:E2E_URL="http://localhost:5173"; node e2e\workflow.e2e.mjs   # real-browser workflow
```

11.1 What the tests prove

Suite	Covers
test_calculations.py	Equations vs hand calculations: S×P sizing, energy, constant-power quadratic, charge/discharge times, lumped thermal closed form; missing cell fields flagged, not invented
test_simulation.py	Initial V = OCV − IR, coulomb counting, energy balance, RC branch vs analytical value, CC-CV respects V_{max} and terminates, constant-power hold, coupled thermal vs closed form, repeatability
test_bms_comparison.py	PASS / WARN / FAIL / NOT_EVALUATED paths, fuse coordination, max safe current at the temperature limit, seeded Monte-Carlo, comparison excludes failing designs
test_prototype_parity_and_api.py	Backend equals stage-1 engine on A/B/C; REST behaviour and labels
test_copilot.py	Scripted model driving the real engine: incorrect claims flagged, negations allowed, repair round, AI cannot set assumptions or use unknown cells, refusals, error mapping, key never exposed
test_storage.py, test_projects_ai.py, test_audit.py	Mongo/SQL stores and fallback, key redaction, project CRUD, grounding of AI explanations, delete rules, rate limits, input/model trace, full workflow
frontend pages.test.jsx	Every page rendered against real backend responses (fixtures), main buttons exercised
frontend simulator.test.jsx	Live simulator parity with backend values, TH-01 at 20 A, dependency propagation, guided tour
e2e/workflow.e2e.mjs	Full workflow in real Chrome: 14/14 steps, 0 errors, screenshots

Verification is not validation. The tests prove the code implements the stated equations correctly. They do not prove the models match real batteries; that needs measured data, which is the first item on the roadmap.

HONEST STATUS

12. Limitations and roadmap

Limitation today	Planned step
Only the illustrative cell is in the database; 7 of 15 checks are NOT_EVALUATED for it	Add a real datasheet cell (template in backend/docs/API.md)
Claude copilot not yet run live (tested with a scripted model)	Live run with saved transcripts (scripts/copilot_smoke.py)
First-order models; single-node thermal	Compare with a public dataset; multi-node thermal with cell-to-cell conduction
No validation against measurements	Supervised single-cell low-voltage bench test
No authentication; Chrome-only e2e	Team accounts; five-user study